

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial intelligence
Practical VII

Roll No.:T031	Name:Ishita Patel
Class:TYIT	Batch:2
Date of Assignment: 01/08/26	Date/Time of Submission:01/08/26

Machine Learning :

1. Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.

a. Load dataset using Pandas.

Code:-

```
# Machine Learning Workflow using Iris Dataset
```

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
# Load Dataset
```

```
df = pd.read_csv("iris.csv")
```

b. Perform basic pre processing (handling missing values or scaling).

- **Load the Dataset**

Code:-

```
print("\n===== DATASET =====")
```

```
print(df)
```

Output:-

```
===== DATASET =====
   sepal_length  sepal_width  petal_length  petal_width  species
0           5.1           3.5           1.4           0.2    setosa
1           4.9           3.0           1.4           0.2    setosa
2           4.7           3.2           1.3           0.2    setosa
3           4.6           3.1           1.5           0.2    setosa
4           5.0           3.6           1.4           0.2    setosa
..          ...           ...           ...           ...     ...
145           6.7           3.0           5.2           2.3  virginica
146           6.3           2.5           5.0           1.9  virginica
147           6.5           3.0           5.2           2.0  virginica
148           6.2           3.4           5.4           2.3  virginica
149           5.9           3.0           5.1           1.8  virginica

[150 rows x 5 columns]
```

- **Display the Dataset**

Code:-

```
# Display Dataset
print("\nFirst 5 Records")
print(df.head())
```

Output:-

```
First 5 Records
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1           3.5           1.4           0.2  setosa
1           4.9           3.0           1.4           0.2  setosa
2           4.7           3.2           1.3           0.2  setosa
3           4.6           3.1           1.5           0.2  setosa
4           5.0           3.6           1.4           0.2  setosa
```

- **Check Dataset Information**

Code:-

```
# Dataset Information
print("\nDataset Information")
print(df.info())
```

Output:-

```
Dataset Information
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   sepal_length    150 non-null    float64
 1   sepal_width     150 non-null    float64
 2   petal_length    150 non-null    float64
 3   petal_width     150 non-null    float64
 4   species         150 non-null    str
dtypes: float64(4), str(1)
memory usage: 6.0 KB
None
```

- **Display Dataset Shape**

Code:-

```
# Dataset Shape
print("\nDataset Shape")
print(df.shape)
```

Output:-

```
Dataset Shape
(150, 5)
```

- **Display Column Names**

Code:-

```
# Column Names
print("\nColumn Names")
```

```
print(df.columns)
```

Output:-

```
Column Names
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
      'species'],
      dtype='str')
```

- **Display Summary Statistics**

Code:-

```
# Summary Statistics
print("\nSummary Statistics")
print(df.describe())
```

Output:-

```
Summary Statistics
      sepal_length  sepal_width  petal_length  petal_width
count      150.000000    150.000000    150.000000    150.000000
mean         5.843333     3.054000     3.758667     1.198667
std          0.828066     0.433594     1.764420     0.763161
min          4.300000     2.000000     1.000000     0.100000
25%          5.100000     2.800000     1.600000     0.300000
50%          5.800000     3.000000     4.350000     1.300000
75%          6.400000     3.300000     5.100000     1.800000
max          7.900000     4.400000     6.900000     2.500000
```

- **Check Data Types**

Code:-

```
# Data Types
print("\nData Types")
print(df.dtypes)
```

Output:-

```
Data Types
sepal_length    float64
sepal_width     float64
petal_length    float64
petal_width     float64
species         str
dtype: object
```

- **Check Missing Values**

Code:-

```
# Missing Values
print("\nMissing Values")
print(df.isnull().sum())
```

Output:-

```
Missing Values
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

- **Handle Missing Values**

Code:-

```
# Handle Missing Values
```

```
df.fillna(df.select_dtypes(include=np.number).mean(), inplace=True)
```

Output:-

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

- **Remove Duplicate Records**

Code:-

```
# Remove Duplicate Records
```

```
df.drop_duplicates(inplace=True)
```

- **Encode Categorical Data (Label Encoding / One-Hot Encoding)**

Code:-

```
# Encode Categorical Data
```

```
encoder = LabelEncoder()
```

```
df["species"] = encoder.fit_transform(df["species"])
```

- **Rename Columns**

Code:-

```
# Rename Columns
```

```
df.rename(columns={  
    "sepal_length": "SepalLength",  
    "sepal_width": "SepalWidth",  
    "petal_length": "PetalLength",  
    "petal_width": "PetalWidth",  
    "species": "Species"  
}, inplace=True)
```

- **Drop Unnecessary Columns**

- **Select Features (Feature Selection)**

Code:-

```
# Drop Unnecessary Columns
```

```
# No unnecessary columns in Iris
```

```
# Feature Selection
```

```
X = df.drop("Species", axis=1)
```

```
y = df["Species"]
```

- **Create New Features (Feature Engineering)**

Code:-

Feature Engineering

```
df["PetalArea"] = df["PetalLength"] * df["PetalWidth"]
```

- **Convert Data Types**

Code:-

```
# Convert Data Types
```

```
df["Species"] = df["Species"].astype(int)
```

Output:-

```
Normalized Data
[[0.22222222 0.625      0.06779661 0.04166667]
 [0.16666667 0.41666667 0.06779661 0.04166667]
 [0.11111111 0.5        0.05084746 0.04166667]
 [0.08333333 0.45833333 0.08474576 0.04166667]
 [0.19444444 0.66666667 0.06779661 0.04166667]]
```

- **Normalize Data (Min-Max Scaling)**

Code:-

```
# Normalize Data
```

```
minmax = MinMaxScaler()
```

```
X_normalized = minmax.fit_transform(X)
```

```
print("\nNormalized Data")
```

```
print(X_normalized[:5])
```

Output:-

```
Normalized Data
[[0.22222222 0.625      0.06779661 0.04166667]
 [0.16666667 0.41666667 0.06779661 0.04166667]
 [0.11111111 0.5        0.05084746 0.04166667]
 [0.08333333 0.45833333 0.08474576 0.04166667]
 [0.19444444 0.66666667 0.06779661 0.04166667]]
```

- **Standardize Data (Standard Scaling)**

Code:-

```
# Standardize Data
```

```
standard = StandardScaler()
```

```
X_standard = standard.fit_transform(X)
```

```
print("\nStandardized Data")
```

```
print(X_standard[:5])
```

Output:-

```
Standardized Data
[[-0.91550949  1.0199714  -1.35773692 -1.33570001]
 [-1.15755966 -0.12808217 -1.35773692 -1.33570001]
 [-1.39960983  0.33113926 -1.41477817 -1.33570001]
 [-1.52063491  0.10152855 -1.30069567 -1.33570001]
 [-1.03653457  1.24958212 -1.35773692 -1.33570001]]
```

- **Split Features and Target Variable**

Code:-

```
# Split Features & Target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_standard,
    y,
    test_size=0.2,
    random_state=42
)
```

Output:-

c. Split dataset into training and testing sets.

Code:-

```
from sklearn.model_selection import train_test_split

# Features (Independent Variables)
X = df.drop("Species", axis=1)    # For Iris Dataset
# X = df.drop("Survived_Status", axis=1) # For Titanic Dataset

# Target (Dependent Variable)
y = df["Species"]                # For Iris Dataset
# y = df["Survived_Status"]      # For Titanic Dataset

# Split the dataset into Training (80%) and Testing (20%)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Display the size of each dataset
print("Training Features Shape:", X_train.shape)
print("Testing Features Shape :", X_test.shape)
print("Training Target Shape :", y_train.shape)
print("Testing Target Shape  :", y_test.shape)
```

Output:-

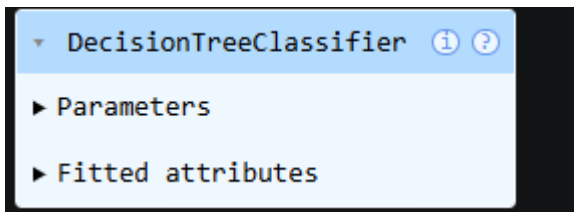
```
Training Features Shape: (117, 5)
Testing Features Shape : (30, 5)
Training Target Shape  : (117,)
Testing Target Shape   : (30,)
```

d. Train a simple classifier.

Code:-

```
# Train Classifier
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
```

Output:-



e. Display training and testing accuracy.

Code:-

```
# Accuracy
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

print("\nTraining Accuracy")
print(accuracy_score(y_train, train_pred))

print("\nTesting Accuracy")
print(accuracy_score(y_test, test_pred))
```

Output:-

```
Training Accuracy
1.0

Testing Accuracy
0.9333333333333333
```

(with another data set)

TatonicDataset:-

Code:-

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

Load Dataset

```
df = pd.read_csv("titanic.csv")
```

Code:-

Display Dataset

```
print(df.head())
```

Output:-

```
survived  pclass    sex  age  sibsp  parch    fare  embarked  class \
0         0         3  male  22.0     1     0   7.2500         S  Third
1         1         1  female  38.0     1     0  71.2833         C  First
2         1         3  female  26.0     0     0   7.9250         S  Third
3         1         1  female  35.0     1     0  53.1000         S  First
4         0         3   male  35.0     0     0   8.0500         S  Third

   who  adult_male  deck  embark_town  alive  alone
0  man         True   NaN  Southampton    no   False
1 woman        False    C  Cherbourg   yes   False
2 woman        False   NaN  Southampton   yes    True
3 woman        False    C  Southampton   yes   False
4  man         True   NaN  Southampton    no    True
```

Code:-

Dataset Information

```
print(df.info())
```

Output:-

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   survived        891 non-null    int64
 1   pclass          891 non-null    int64
 2   sex             891 non-null    str
 3   age            714 non-null    float64
 4   sibsp          891 non-null    int64
 5   parch          891 non-null    int64
 6   fare           891 non-null    float64
 7   embarked       889 non-null    str
 8   class          891 non-null    str
 9   who            891 non-null    str
10   adult_male     891 non-null    bool
11   deck          203 non-null    str
12   embark_town    889 non-null    str
13   alive         891 non-null    str
14   alone         891 non-null    bool
dtypes: bool(2), float64(2), int64(4), str(7)
memory usage: 92.4 KB
None
```

Code:-

Dataset Information

```
print(df.info())
```

Output:-

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   survived        891 non-null    int64
 1   pclass          891 non-null    int64
 2   sex             891 non-null    str
 3   age            714 non-null    float64
 4   sibsp          891 non-null    int64
 5   parch          891 non-null    int64
 6   fare           891 non-null    float64
 7   embarked       889 non-null    str
 8   class          891 non-null    str
 9   who            891 non-null    str
10   adult_male     891 non-null    bool
11   deck          203 non-null    str
12   embark_town    889 non-null    str
13   alive         891 non-null    str
14   alone         891 non-null    bool
dtypes: bool(2), float64(2), int64(4), str(7)
memory usage: 92.4 KB
None
```

Code:-


```
# Shape
print("Shape:", df.shape)
```

Output:-

```
Shape: (891, 15)
```

Code:-

```
# Column Names
print(df.columns)
```

Output:-

```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
      'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
      'alive', 'alone'],
      dtype='str')
```

Code:-

```
# Summary Statistics
print(df.describe(include='all'))
```

Output:-

	survived	pclass	sex	age	sibsp	parch	\
count	891.000000	891.000000	891	714.000000	891.000000	891.000000	
unique	NaN	NaN	2	NaN	NaN	NaN	
top	NaN	NaN	male	NaN	NaN	NaN	
freq	NaN	NaN	577	NaN	NaN	NaN	
mean	0.383838	2.308642	NaN	29.699118	0.523008	0.381594	
std	0.486592	0.836071	NaN	14.526497	1.102743	0.806057	
min	0.000000	1.000000	NaN	0.420000	0.000000	0.000000	
25%	0.000000	2.000000	NaN	20.125000	0.000000	0.000000	
50%	0.000000	3.000000	NaN	28.000000	0.000000	0.000000	
75%	1.000000	3.000000	NaN	38.000000	1.000000	0.000000	
max	1.000000	3.000000	NaN	80.000000	8.000000	6.000000	

	fare	embarked	class	who	adult_male	deck	embark_town	alive
count	891.000000	889	891	891	891	203	889	891
unique	NaN	3	3	3	2	7	3	2
top	NaN	S	Third	man	True	C	Southampton	no
freq	NaN	644	491	537	537	59	644	549
mean	32.204208	NaN	NaN	NaN	NaN	NaN	NaN	NaN
std	49.693429	NaN	NaN	NaN	NaN	NaN	NaN	NaN
min	0.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25%	7.910400	NaN	NaN	NaN	NaN	NaN	NaN	NaN
50%	14.454200	NaN	NaN	NaN	NaN	NaN	NaN	NaN
75%	31.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN
max	512.329200	NaN	NaN	NaN	NaN	NaN	NaN	NaN
...								
25%	NaN							
50%	NaN							
75%	NaN							
max	NaN							

Code:-

```
# Data Types
print(df.dtypes)
```

Output:-

```
survived      int64
pclass        int64
sex           str
age           float64
sibsp         int64
parch         int64
fare          float64
embarked      str
class         str
who           str
adult_male    bool
embark_town   str
alive         str
alone         bool
dtype: object
```

Code:-

```
# Missing Values
print(df.isnull().sum())
```

Output:-

```
survived      0
pclass        0
sex           0
age           0
sibsp         0
parch         0
fare          0
embarked      0
class         0
who           0
adult_male    0
embark_town   0
alive         0
alone         0
dtype: int64
```

Code:-

```
# Handle Missing Values
df["age"] = df["age"].fillna(df["age"].mean())
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])
df["embark_town"] = df["embark_town"].fillna(df["embark_town"].mode()[0])
```

Code:-

```
# Drop deck because it has too many missing values
df.drop(columns=["deck"], inplace=True)
```

Code:-

```
# Remove Duplicates
df.drop_duplicates(inplace=True)
```

Code:-

```
# Encode Categorical Columns
```

```
le = LabelEncoder()
```

```
categorical = ["sex","embarked","class","who","alive","embark_town","alone","adult_male"]
```

```
for col in categorical:
```

```
    df[col] = le.fit_transform(df[col])
```

Code:-

```
# Rename Columns
```

```
df.rename(columns={  
    "survived":"Survived",  
    "pclass":"PassengerClass"  
}, inplace=True)
```

Code:-

```
# Feature Engineering
```

```
df["FamilySize"] = df["sibsp"] + df["parch"] + 1
```

Code:-

```
# Feature Selection
```

```
X = df.drop("Survived", axis=1)
```

```
y = df["Survived"]
```

Code:-

```
# Standard Scaling
```

```
scaler = StandardScaler()
```

```
X = scaler.fit_transform(X)
```

Code:-

```
# Train Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

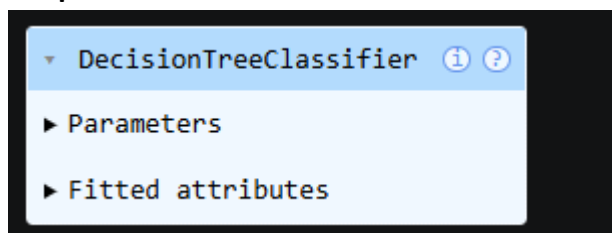
Code:-

```
# Train Model
```

```
model = DecisionTreeClassifier(random_state=42)
```

```
model.fit(X_train, y_train)
```

Output:-



Code:-

```
# Accuracy
```

```
train_accuracy = accuracy_score(y_train, model.predict(X_train))
```

```
test_accuracy = accuracy_score(y_test, model.predict(X_test))
```

```
print("\nTraining Accuracy :", train_accuracy)  
print("Testing Accuracy :", test_accuracy)
```

Output:-

```
Training Accuracy : 1.0  
Testing Accuracy : 1.0
```